



Algoritmos y Estructuras de Datos

1° Año

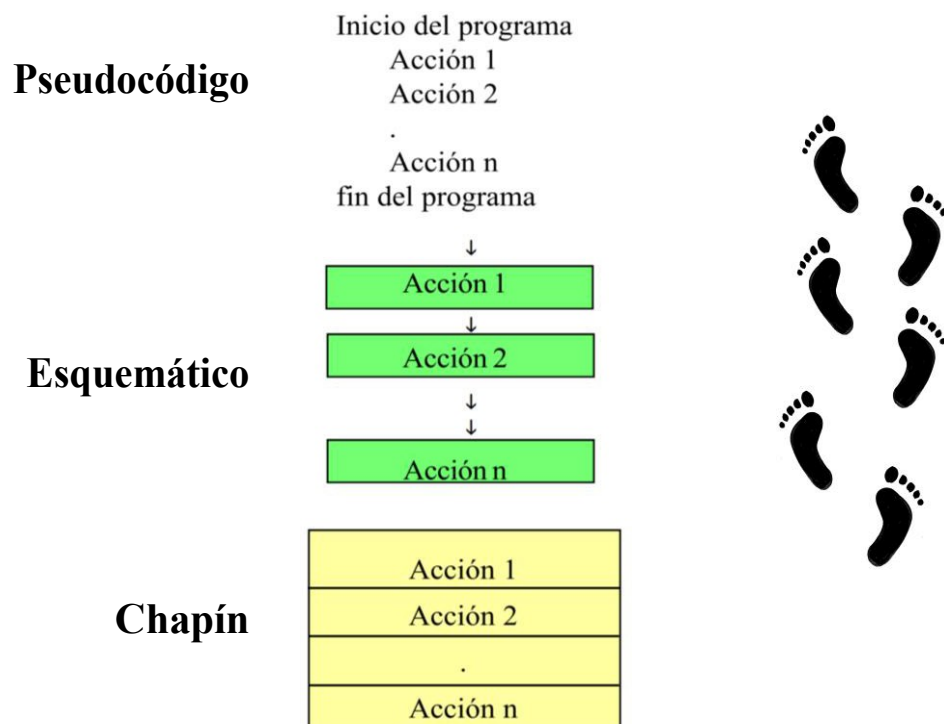
Ingeniería en Sistemas de Información

Estructuras de Control

*Si podemos identificar como actúa nuestro cerebro,
podremos diagramar y luego programar*

SECUENCIA

La estructura de secuencia es un tipo de estructura de control, también llamada **estructura secuencial**. Hace referencia al orden de ejecución de las instrucciones que se hace de forma secuencial, o sea, una instrucción después de la otra (paso a paso). Las formas de representar gráficamente las estructuras de control pueden ser diversas, les presentamos modelos de visualizaciones gráficas.



ESTRUCTURAS DE CONTROL Repetitivas y Decisorias

Estas estructuras se usan en todos los programas que tengan que repetir un conjunto de instrucciones cumpliendo con una condición lógica ó cuando se debe elegir un grupo de sentencias entre varias posibles.

Cuando **NO** se conoce el número preciso de repetición se lo llama **Bucle condicional**.

Cuando **SI** se conoce el número de repetición se lo llama **Bucle incondicional**.

Cuando se presente la necesidad de efectuar una **decisión lógica** para ver cuál es la acción a realizar entre dos posibles se denomina **ejecución condicional** y cuando se elige un grupo de sentencias entre varios grupos posibles se llama **selección**.

En cualquiera de las estructuras de control se pueden utilizar operadores relacionales (= < > <= >= <> == !=) combinados ó no, con operadores lógicos (OR , AND , NOT).

DECISION
(entre dos alternativas)
SI

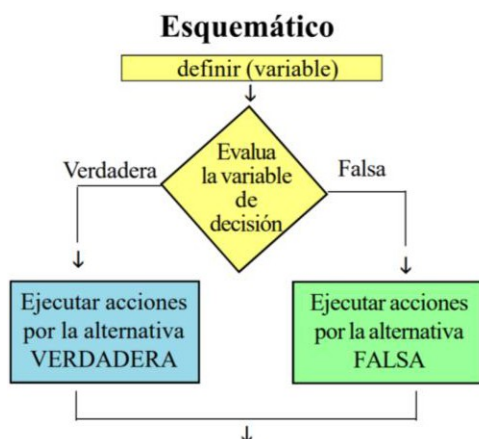
IF : else : (Python)



*Tengo que elegir 1 puerta: si o si,
no puedo pasar por las dos
Como verán las dos existen
pero se opta por una
en tiempo de ejecución*

Pseudocódigo

definir (variable)
SI variable ENTONCES acciones
verdaderas SINO acciones
falsas



Chapín

definir (variable)	
verdadera	falsa
Av1	Af1
Av2	Af2
.	.
Avk	Afn

Python

IF expresión-lógica:
sentencia-verdadera

Los tabulados tienen importancia para que se interprete lo que debe hacer. En este caso solo se programa por verdadero

IF expresión-lógica:
sentencia-verdadera
else:
sentencia-falsa

Los tabulados se ubican según se trabaje por verdadero, o se trabaje por falso respondiendo al ELSE

IF expresión-lógica:
sentencia-verdadera
elif:
sentencia-falsa

Este modelo es una contracción de else if

REPETICION INCONDICIONAL

(cantidad conocida de veces)

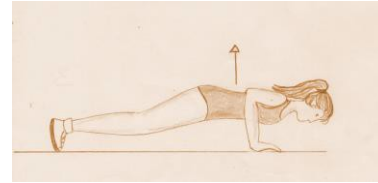
PARA

FOR IN : (Python)

Pseudocódigo

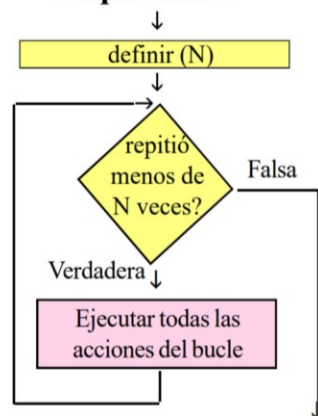
```

definir (N)
hacer N Veces
    acción 1
    acción 2
    .
    acción k
  
```

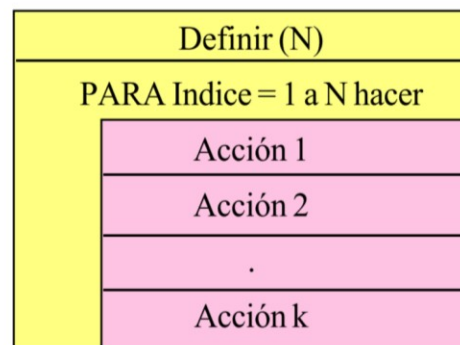


*Hacerlo 10 veces sin parar
conozco de antemano la
cantidad de veces que lo
debo hacer
Las repeticiones son
exactas*

Esquemático



Chapín



La variable de control debe ser una variable de cualquier tipo **menos real** dado que cuenta cantidades exactas, se la puede usar dentro del proceso de repetición sin modificarla, pero finalizada la repetición, no puede utilizársela sin redefinirla.

Se utiliza en el ciclo for para delimitar la cantidad de vueltas del ciclo.

Rango es un tipo de secuencia numérica designada como *rango* (o *range*), de forma que el ciclo itere sobre ella.

Es una sucesión *fija* de valores numéricos en un intervalo definido, que puede ser creada fácilmente con la función *range()*.

Formato general (inicio, fin, paso):

Ejemplo:

<code>range(0,5)</code>	#	valores	generados	{0, 1, 2, 3, 4}
<code>range(5)</code>	#	valores	generados	{0, 1, 2, 3, 4}
<code>range(1,6)</code>	#	valores	generados	{1, 2, 3, 4, 5}
<code>range(5,0,-1)</code>	#	valores	generados	{5, 4, 3, 2, 1}
<code>range(0,10,2)</code>	#	valores	generados	{0, 2, 4, 6, 8}

Si no se pone el paso se asume que es 1, es decir, la variable índice se incrementa de uno en uno por cada ciclo.

Si no se pone el valor de inicio arranca en 0 cero

Python

FOR variable de control **IN rango** (valor): los tabulados tienen importancia para reconocer todas sus sentencias
Sentencias

FOR variable de control **IN** (valor1, valor2): en lugar de un rango la variable de control toma los valores declarados entre ()
Sentencias

Ejemplo

```
for W in range(0, 25):  
    print (W)
```

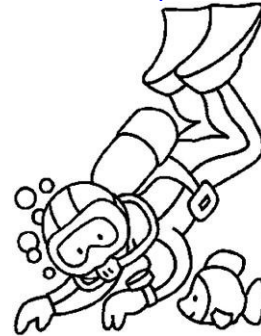
REPETICION PRE - CONDICIONAL

(cantidad desconocida de veces)

MIENTRAS

WHILE : (Python)

Mientras tenga aire
te acompaño

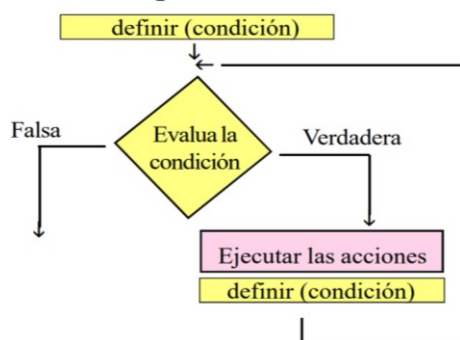


primero verifico si tengo aire,
puede ser que no lo acompañe
nunca si el tanque está vacío

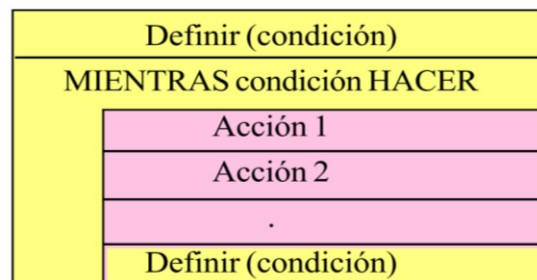
Pseudocódigo

```
definir (condición)
MIENTRAS condición HACER
    acción 1
    acción 2
    .....
definir (condición)
```

Esquemático



Chapín



Las sentencias se ejecutan repetidamente en tanto la expresión lógica sea verdadera. Primero, verifica y luego ejecuta, por lo cual puede no ejecutarse el grupo de sentencias si en la primera validación salió por falso. Es fundamental mantener el tabulado para todas las instrucciones que deben repetirse. Debe tener una sentencia de asignación {externa (lectura) o interna (=)} de la variable interviniente en la decisión (condición), externa a la estructura al comienzo para hacer la primera validación y otra interna (en la estructura) para las posteriores y que no entre en loop.

WHILE expresión lógica:
sentencia

los tabulados tienen importancia para
reconocer todas sus sentencias

Ejemplo de cuenta regresiva en Python

```
Input (n)
while n > 0:
    print n
    n = n-1
print "¡Despegue!"
```

REPETICION POS-CONDICIONAL

(cantidad desconocida de veces)

REPETIR - HASTA / REPEAT – UNTIL



*Hasta que mis piernas se cansen
(tengo que saltar al menos una vez para saber si estoy cansado)*

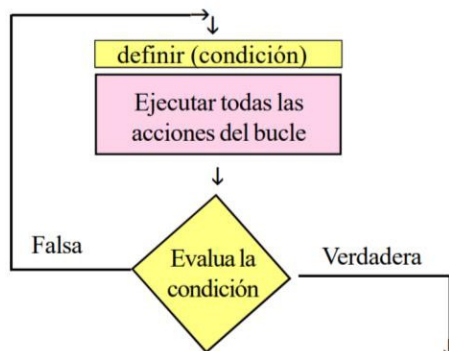
Pseudocódigo

```

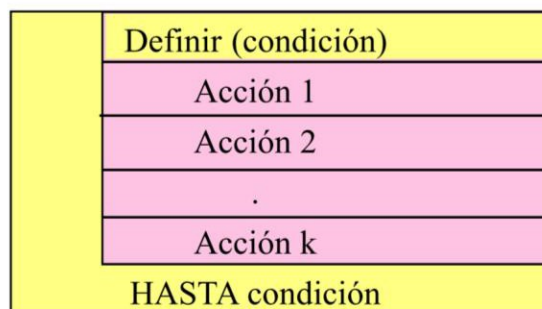
REPETIR
    definir (condición)
    acción 1
    acción 2
    .
    acción k
HASTA condición
  
```

Python no posee esta estructura de control. Es reemplazada por el **mientras**. Pero la usaremos todas las veces que queramos en el diagrama de chapín.

Esquemático



Chapín



La secuencia de sentencias se ejecutará repetidamente hasta que la expresión lógica (condición) sea verdadera, por lo cual primero ejecuta y luego compara (a la inversa del WHILE), ejecutándose al menos una vez todas las sentencias.

DECISION o SELECCION

(ejecución condicional, entre varias alternativas)

SELECCION MULTIPLE / CASE



Cada camino me llevará a un destino diferente, solo uno de ellos puedo elegir.

Pseudocódigo

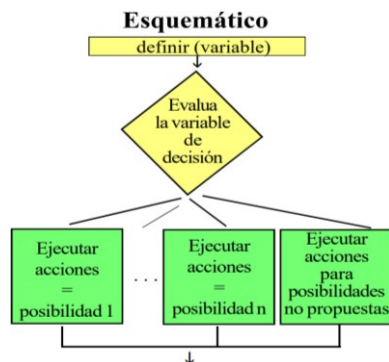
```

definir (condición)
en CASO condición SEA IGUAL
    posibilidad 1: acciones posibilidad 1
    posibilidad 2: acciones posibilidad 2
    .
    posibilidad n: acciones posibilidad n
    otras: acciones para otros casos
fin de CASO DE SELECCION
  
```

Python

```

match variable:
    case posibilidad-1:
        action-1
    case posibilidad-2:
        action-2
    case posibilidad-3:
        action-3
    case _:
        action-default
  
```



Chapín

definir (variable)				
variable				
posibilidad 1	posibilidad 2	...	posibilidad n	otras posibilidades
acciones para variable = posibilidad 1	acciones para variable = posibilidad 2	...	acciones para variable = posibilidad n	acciones de posibilidades no previstas

Permite la selección de un grupo particular de sentencias entre varios grupos posibles, la selección se hace sobre el valor de una expresión denominada selector.

La expresión no puede ser real, generalmente es una variable simple.

Cada rótulo case, es uno de los valores posibles de la expresión, no deben respetar un orden determinado. Es importante el tabulado dentro de cada rótulo para que identifique todas las sentencias. Se permiten sentencias nulas.

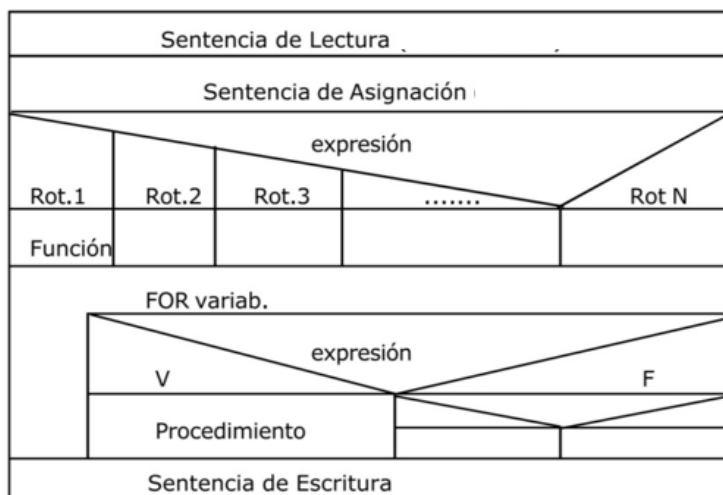
Si el valor de la expresión no coincide con ningún rótulo, la acción a realizar estará indefinida, aunque se pueden establecer acciones por default.

CASE puede ser una alternativa de la estructura IF anidada.

A diferencia de ésta, las comparaciones se hacen SOLO por igualdad.

Anidamiento de estructuras de control

Todas las estructuras de control se pueden anidar, ya sean del mismo tipo o distintas, mientras se mantenga la lógica secuencial de lo que se está diagramando.

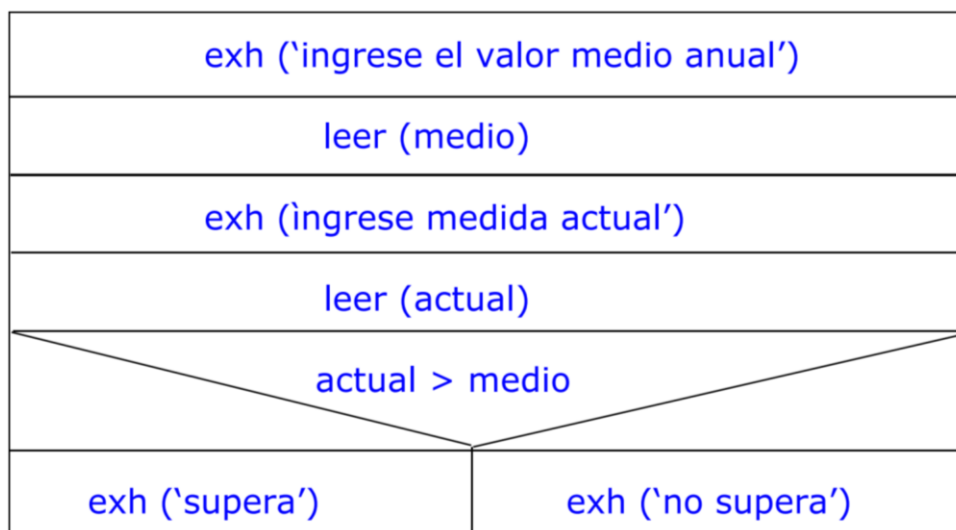


Ejemplos

Observen, según la redacción del enunciado, como se utilizan las diferentes estructuras de control. Cada una responde a un pedido específico

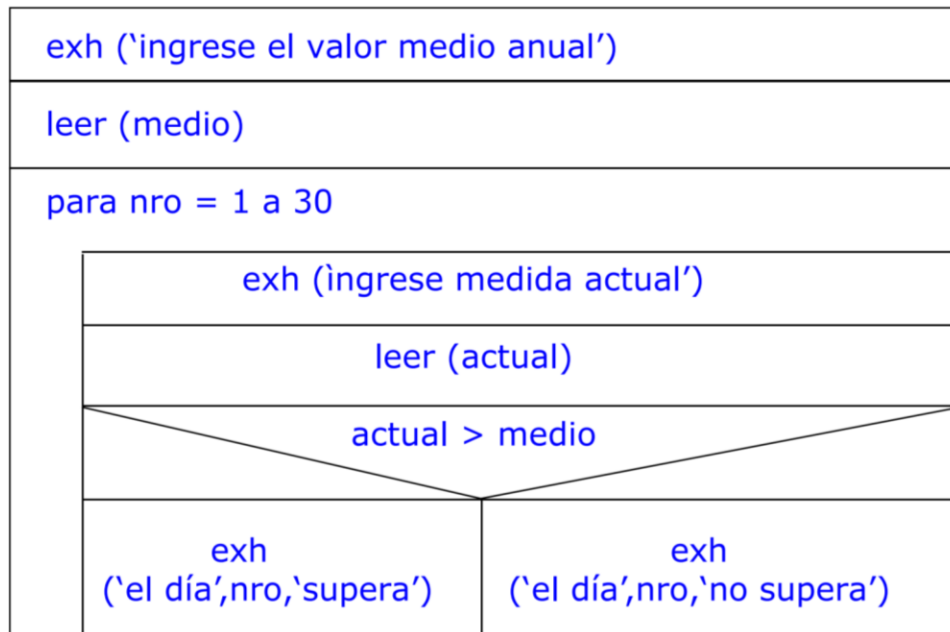
- 1) Dada **la altura** actual del río, determinar **si supera** el valor de la media anual.

ALTURA



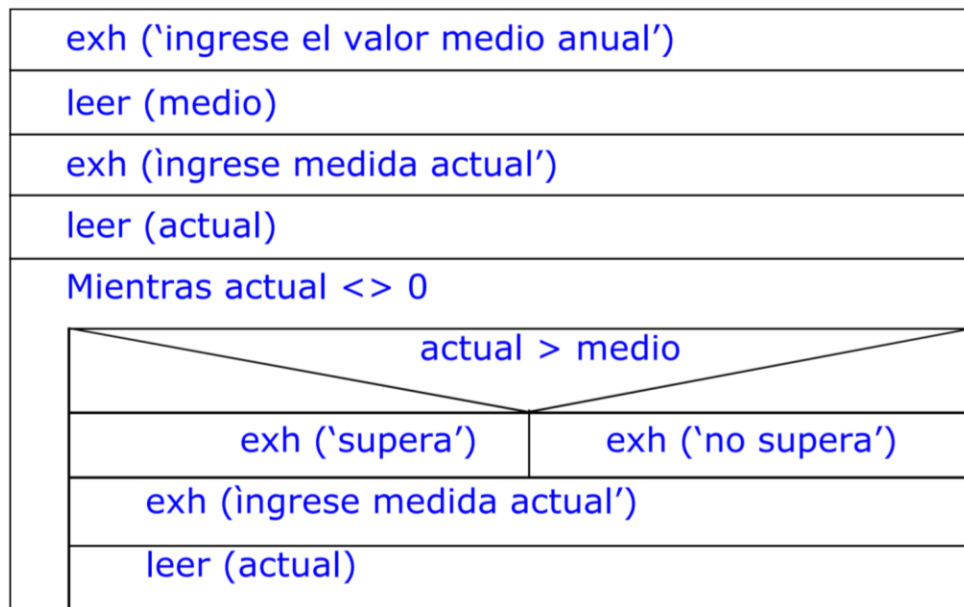
- 2) Determinar **durante 30** días, si **la altura** actual del río **supera** la media anual.

ALTURAX30



- 3) Dado el ingreso **por día**, de la altura actual del río, considerando **altura = 0** como **fin de datos**, determinar día por día, **si supera** el valor de la media anual.

ALTURAXDIA



- 4) Dado el ingreso **por día**, de la altura actual del río, considerando **altura = tope como último dato** a procesar, determinar día por día, **si supera** el valor de la media anual.

ALTURAXTOPE

exh ('ingrese el valor tope')	
leer (tope)	
exh ('ingrese el valor medio anual')	
leer (medio)	
	exh ('ingrese medida actual')
	leer (actual)
	actual > medio
	exh ('supera')
	exh ('no supera')
hasta actual = tope	

- 5) Dado el ingreso de **la altura** actual del río, compararla con **las últimas 4 estadísticas** y establecer con cuál de ellas coincide

ALTURAXESTADISTICA

exh ('ingrese estadística 1')				
leer (e1)				
exh ('ingrese estadística 2')				
leer (e2)				
exh ('ingrese estadística 3')				
leer (e3)				
exh ('ingrese estadística 4')				
leer (e4)				
exh ('ingrese el valor actual')				
leer (actual)				
actual				
e1	e2	e3	e4	otros
exh	exh	exh	exh	exh
('coincide con 1')	('coincide con 2')	('coincide con 3')	('coincide con 4')	('NO coincide')